UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ Accredited University

Student Name : **Priyanka Chandwani** **UID:** **25MCI10122**
Branch **: MCA (AI & ML)** **Section/Group :** **25MAM-1-A**
Semester **: 2nd** **Date of Performance:** **24.02.2026**
Subject Name : **Technical Training** **Subject Code :** **25CAP-652**

Experiment 5:

Experiment Aim: To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic. **(Company Tags: Infosys, Wipro, TCS, Capgemini)**

Tools used: PostgreSQL

Objectives

- **Sequential Data Access:** To understand how to fetch rows one by one from a result set using cursor mechanisms.
- **Row-Level Manipulation:** To perform specific operations or calculations on individual records that require conditional procedural logic.
- **Resource Management:** To learn the lifecycle of a cursor: Declaring, Opening, Fetching, and importantly, Closing and Deallocating to manage system memory.
- **Exception Handling:** To handle cursor-related errors and performance considerations during large-scale data iteration.

Experiment Steps:

Table Creation and Data Storing-

```
CREATE TABLE Employee (
   EmpID INT PRIMARY KEY,
   EmpName VARCHAR(50),
   Experience INT,
   PerformanceRating INT,
   Salary DECIMAL(10,2)
);

INSERT INTO Employee VALUES
(1,'Amit',3,4,30000),
(2,'Neha',6,5,55000),
(3,'Rahul',1,3,20000),
(4,'Pooja',8,5,70000);

select * from employee;
```

	empid [PK] integer	empname character varying (50)	experience integer	performancerating integer	salary numeric (10,2)
1	1	Amit	3	4	30000.00
2	2	Neha	6	5	55000.00
3	3	Rahul	1	3	20000.00
4	4	Pooja	8	5	70000.00

Step 1: Implementing a Simple Forward-Only Cursor

Creating a cursor to loop through an Employee table and print individual records.

```
DO $$

DECLARE

   emp_cursor CURSOR FOR SELECT * FROM Employee;
```

```
    emp_row RECORD;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO emp_row;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE '% % %', emp_row.EmpID, emp_row.EmpName, emp_row.Salary;
    END LOOP;
    CLOSE emp_cursor;
END $$;
```

```
NOTICE:  1 Amit 30000.00
NOTICE:  2 Neha 55000.00
NOTICE:  3 Rahul 20000.00
NOTICE:  4 Pooja 70000.00
DO

Query returned successfully in 299 msec.
```

Step 2: Complex Row-by-Row Manipulation

Using a cursor to update salaries based on a dynamic "Experience-to-Performance" ratio logic.

```
DO $$
DECLARE
    emp_cursor CURSOR FOR SELECT * FROM Employee;
    emp_row RECORD;
```

```
    increment INT;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO emp_row;
        EXIT WHEN NOT FOUND;
        increment := emp_row.Experience * emp_row.PerformanceRating * 1000;
        UPDATE Employee
        SET Salary = Salary + increment
        WHERE EmpID = emp_row.EmpID;
    END LOOP;
    CLOSE emp_cursor;
END $$;
select * from employee;
```

	empid [PK] integer	empname character varying (50)	experience integer	performancerating integer	salary numeric (10,2)
1	1	Amit	3	4	42000.00
2	2	Neha	6	5	85000.00
3	3	Rahul	1	3	23000.00
4	4	Pooja	8	5	110000.00

Step 3: Exception and Status Handling

Ensuring the cursor handles empty result sets or termination signals gracefully.

```
DO $$
```

```
DECLARE
    emp_cursor CURSOR FOR SELECT * FROM Employee;
    emp_row RECORD;
BEGIN
    OPEN emp_cursor;


    LOOP
        FETCH emp_cursor INTO emp_row;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE '% % %', emp_row.empid, emp_row.empname, emp_row.salary;
    END LOOP;
    CLOSE emp_cursor;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Cursor Error Occurred';
END $$;
```

```
NOTICE:  1 Amit 42000.00
NOTICE:  2 Neha 85000.00
NOTICE:  3 Rahul 23000.00
NOTICE:  4 Pooja 110000.00
DO

Query returned successfully in 288 msec.
```

Outcomes:

- **Cursor Implementation:** Students will be able to design, implement, and manage cursors to solve row-wise processing problems.
- **Lifecycle Mastery:** Students will demonstrate the correct syntax for declaring, opening, fetching, and closing cursors.
- **Error Prevention:** Students will understand how to properly handle row-by-row processing exceptions and prevent memory leaks via deallocation.
- **Analytical Thinking:** Students will be able to apply cursor-based logic to solve real-world scenarios like multi-level payroll adjustments or data migrations.